



Design and Analysis of Algorithms

Vandana M L

Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Mathematical Analysis of Non-recursive Algorithms

Slides courtesy of **Anany Levitin**

Vandana M L

Department of Computer Science & Engineering

Design and Analysis of Algorithms

Time Efficiency of Non-recursive Algorithms

Steps in mathematical analysis of non-recursive algorithms:

- Decide on parameter n indicating input size
- Identify algorithm's basic operation
- Check whether the number of times the basic operation is executed depends only on the input size n . If it also depends on the type of input, investigate worst, average, and best case efficiency separately.
- Set up summation for $C(n)$ reflecting the number of times the algorithm's basic operation is executed.
- Simplify summation using standard formulas

Design and Analysis of Algorithms

Useful Summation Formulas and Rules

$$\sum_{l \leq i \leq u} 1 = 1 + 1 + \dots + 1 = u - l + 1$$

$$\sum_{1 \leq i \leq n} i = 1 + 2 + \dots + n = n(n+1)/2$$

$$\sum_{1 \leq i \leq n} i^2 = 1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6$$

$$\sum_{0 \leq i \leq n} a^i = 1 + a + \dots + a^n = (a^{n+1} - 1)/(a - 1) \text{ for any } a \neq 1$$

$$\sum (a_i \pm b_i) = \sum a_i \pm \sum b_i \quad \sum c a_i = c \sum a_i$$

$$\sum_{l \leq i \leq u} a_i = \sum_{l \leq i \leq m} a_i + \sum_{m+1 \leq i \leq u} a_i$$

$$\sum_{i=l}^u 1 = (u - l + 1)$$

AP $S_n = n/2 [2a + (n-1)d].$

GP $S_n = a(r^n - 1) / (r - 1)$

Design and Analysis of Algorithms

Example 1: Finding Max Element in a list

Algorithm *MaxElement* ($A[0..n-1]$)

//Determines the value of the largest element
in a given array

//Input: An array $A[0..n-1]$ of real numbers

//Output: The value of the largest element in A

maxval $\leftarrow A[0]$

for $i \leftarrow 1$ to $n-1$ do

 if $A[i] > \text{maxval}$

 maxval $\leftarrow A[i]$

return maxval

- The basic operation- comparison
- Number of comparisons is the same for all arrays of size n .
- Number of comparisons

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

Design and Analysis of Algorithms

Example 2: Element Uniqueness Problem

Algorithm *UniqueElements* ($A[0..n-1]$)

//Checks whether all the elements in a given array are distinct

//Input: An array $A[0..n-1]$

//Output: Returns true if all the elements in A are distinct and false otherwise

```
for i  $\leftarrow$  0 to n - 2 do
    for j  $\leftarrow$  i + 1 to n - 1 do
        if  $A[i] = A[j]$  return false
return true
```

Best-case:

If the two first elements of the array are the same
No of comparisons in Best case = 1 comparison

Worst-case:

- Arrays with no equal elements
- Arrays in which only the last two elements are the pair of equal elements

Design and Analysis of Algorithms

Example 2: Element Uniqueness Problem

$$\begin{aligned}C_{\text{worst}}(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) \\&= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} \\&= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2\end{aligned}$$

Best-case: 1 comparison

Worst-case: $n^2/2$ comparisons

$$T(n)_{\text{worst case}} = O(n^2)$$

Design and Analysis of Algorithms

Example 3: Matrix Multiplication

Algorithm *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

//Multiplies two square matrices of order n by the definition-based algorithm

//Input: two n -by- n matrices A and B

//Output: Matrix $C = AB$

for $i \leftarrow 0$ to $n - 1$ do

 for $j \leftarrow 0$ to $n - 1$ do

$C[i, j] \leftarrow 0.0$

 for $k \leftarrow 0$ to $n - 1$ do

$C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$

return C

$M(n) \in \Theta(n^3)$

Design and Analysis of Algorithms

Example 3: Matrix Multiplication

Algorithm *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

//Multiplies two square matrices of order n by the definition-based algorithm

//Input: two n-by-n matrices A and B

//Output: Matrix $C = AB$

for $i \leftarrow 0$ to $n - 1$ do

 for $j \leftarrow 0$ to $n - 1$ do

$C[i, j] \leftarrow 0.0$

 for $k \leftarrow 0$ to $n - 1$ do

$C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$

return C

$M(n) \in \Theta(n^3)$

In the MaxElement algorithm, which finds the largest value in the array why is the comparison $A[i] > \text{maxval}$ specifically selected as the basic operation?

- A) It has a higher computational cost per execution than the assignment operation
- B) It only executes when a new maximum value is found.
- C) It executes a fixed number of times, independent of the actual data in the array.
- D) It takes more constant time to execute than other operations in the algorithm.

Answer

- C) It executes a fixed number of times, independent of the actual data in the array.

In the MaxElement algorithm, which finds the largest value in an unsorted array of n elements, how many comparison operations are performed between array elements?

- a) n
- b) $n-1$
- c) $n(n-1)/2$
- d) $\Theta(n^2)$

Answer : B

In the element uniqueness problem, what characteristic defines the worst-case input?

- A) All elements are identical
- B) All elements are distinct
- C) The largest element appears first
- D) The smallest element appears last

Answer

- B) All elements are distinct

Why does the Binary(n) algorithm that is used to compute the number of digits in a binary equivalent of a given number have time complexity $O(\log n)$?

- A) The loop body executes a constant number of times
- B) The input size decreases by a constant factor each iteration
- C) The loop condition is evaluated more times than the body
- D) The algorithm uses division instead of subtraction

Answer

- B) The input size decreases by a constant factor each iteration

The total number of scalar multiplications performed in the standard $n \times n$ matrix multiplication algorithm is:

- a) n^2
- b) $n^2 - n$
- c) n^3
- d) $\frac{n^3}{2}$

Answer : C)



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu